Kubernetes Architechture

Kubernetes Cluster

Master Node

Controls the cluster &manages its resources

ETCD

a consistent and highly-available key-value store used as Kubernetes' backing store for all cluster data including cluster state, cluster configurations and others.

control plane

Kube Controller Manager

Watches and controls the worker nodes overall. Ensure that we get the correct number of pods up and running.

Kubernetes API server
an entry point where user ask to create a new POD

Cloud Controller Manager

1. Basically just translates instructions to AWS or Azure or whatever you're using.

2. lets us link our cluster to a specific cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with our cluster.

scheduler provides this information to the cube API server

Scheduler
Used to assign PODs to the worker nodes.

1. It will check which **nodes have enough capacity to launch the new POD**

2. Watching our pods and choosing worker nodes on which new pods should be created on, in case a new pod needs to be added because a pod got unhealthy and went down, or because of scaling, we needed to create a new pod.

Worker Node 1

(One Computer/Machine/Virtual Instance)

1. Worker can have one or more PODs
2. Worker node is managed by Master node
3. Worker is just a machine somewhere on the internet with a certain amount of CPU and memory

Get info from Master Node from where should be POD running & how manage it

kubelet

Kubelet agent
Deploy & manage PODs inside the worker node

k-proxy

Kube-Proxy POD to POD communication & expose application

docker

Required by the Pods to create and run the Containers

POD 1
Container

POD 2
Container 1
Container 2

1. PODs ar created & managed by Kubernets.
2. POD hosts one or more application conainers & their resources (volumes, IP, Run config)

Worker Node 2

(One Computer/Machine/Virtual Instance)

1. Worker can have one or more PODs
2. Worker node is managed by Master node
3. It's just a machine somewhere on the internet with a certain amount of CPU and memory

kubelet

It's basically a software service running on that machine, on the Worker Node machine, which does the actual communication with the Master Node, so that the Master Node is able to control the Pods on this Worker Node.

k-proxy

1. Responsible for handling incoming and outgoing traffic to insure that everything is working as desired

2. Only allowed traffic is able to reach the Pods

3. Only allowed traffic is able to leave the Worker Node

docker

Required by the Pods to create and run the Containers

POD 1
Container

POD 2
Container